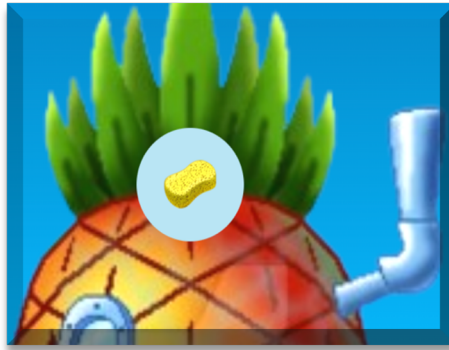




SpongeBob Inspired

A Web-Based Image Cartoonification System

Digital Image Processing project

2025-2026

Students:

Raghad Fayeze
Meteabh Aldawsri
Hanan Abdallah
Nouf Alqassem
Lama Alruwais

ID:

444500629
443500038
443520329
443500106
443500170

Supervisor:

Dr.Dimah Almani

Abstract

The SpongeBob Inspired project is designed to transform ordinary photos into cartoon-style images inspired by the SpongeBob SquarePants character, using classical digital image processing techniques. The system aims to provide users with a fun and creative experience through a web platform that allows uploading images or capturing them directly from the camera. By applying processes such as bilateral filtering, edge detection, and color quantization, the system generates vibrant cartoon results that reflect SpongeBob's cheerful style. The project demonstrates how technology and art can be combined to create an engaging and interactive digital experience accessible to everyone.

موجز المشروع باللغة العربية

يهدف مشروع «SpongeBob Inspired» إلى تحويل الصور العادية إلى صور كرتونية مستوحاة من شخصية سبونج بوب باستخدام تقنيات معالجة الصور الرقمية الكلاسيكية. يسعى النظام إلى توفير تجربة ممتعة وإبداعية للمستخدمين من خلال منصة ويب تتيح لهم رفع الصور أو التقاطها مباشرة عبر الكاميرا. ومن خلال تطبيق عمليات مثل التنعيم الثنائي واكتشاف الحواف وتبسيط الألوان، ينتج النظام صورًا كرتونية نابضة بالحياة تعكس الطابع المرح لشخصية سبونج بوب. يبرز المشروع كيف يمكن الجمع بين التقنية والفن لإنشاء تجربة رقمية تفاعلية وجذابة متاحة للجميع.

Table of Contents

Topic	page
1. Project Idea	4
2. Objective	4
3. Background	4
4. Motivation	4
5. Methodology	5
6. System Design	6
7. Implementation Details	6
8. Results and Discussion	8
9. Conclusions and Future Work	10
References	11

Table of Figures

Figure	Title	page
Figure 1	Uploading an image from the device	7
Figure 2	Clicking the “Convert to Cartoon” button to process the uploaded image	8
Figure 3	The cartoonified result of the uploaded image and controlling the intensity	8
Figure 4	Capturing an image directly from the camera and displaying the cartoonified result	8

1. Project Idea

SpongeBob Inspired is a creative image processing project designed to transform real-life photos into cartoon-style images inspired by the popular animation SpongeBob SquarePants. The system applies image processing techniques to simplify details, highlight outlines, and use a playful color palette dominated by bright yellow tones — replicating the cheerful SpongeBob style. The project combines technology and creativity through a web-based interface that allows users to either upload an image or capture one directly using their device's camera, then instantly generate a cartoonified result in SpongeBob style.

2. Objectives

The main objective of this project is to combine technology and creativity to provide a joyful and engaging experience that allows users to transform real photos into SpongeBob-style cartoon images.

The specific objectives are:

1. To design a simple web application that allows photo upload and direct camera capture.
2. To apply classical image processing techniques such as bilateral filtering and edge detection.
3. To make the process fun, fast, and accessible for all users.
4. To encourage creativity and self-expression through interactive technology.

3. Background

Cartoon-style image generation has gained attention in both research and entertainment due to its aesthetic and artistic value. The process involves simplifying textures, enhancing edges, and creating flat color regions to mimic animation artwork. Tomasi and Manduchi [1] introduced the bilateral filtering technique, which smooths image areas while preserving edges, making it ideal for cartoon-like rendering. Canny [2] proposed a classic edge detection algorithm that remains widely used to extract clear object boundaries. In a more recent approach, Chen et al. [3] developed CartoonGAN, which uses deep learning to stylize photos based on cartoon examples. While deep learning offers advanced stylization, it often requires large datasets and heavy computation. In contrast, SpongeBob Inspired applies efficient and lightweight OpenCV techniques suitable for real-time web applications, achieving visually appealing results that emulate the joyful SpongeBob aesthetic.

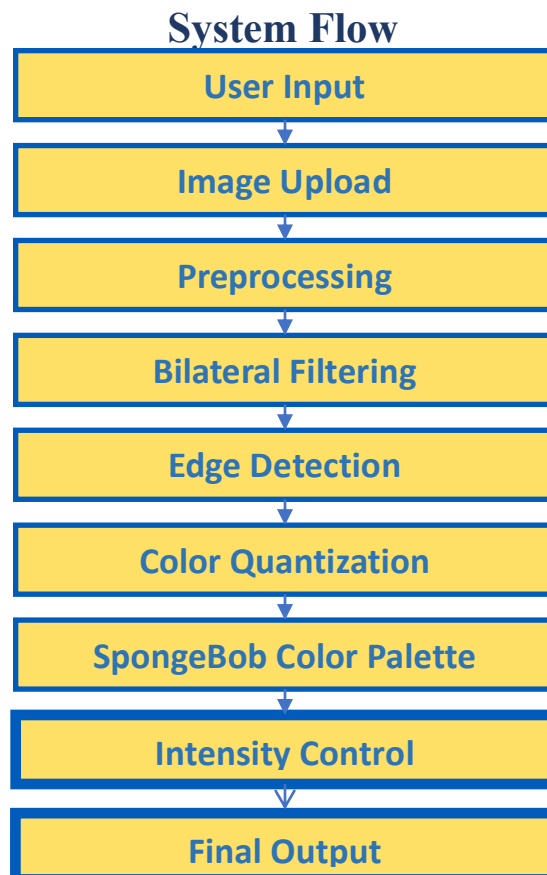
4. Motivation

Cartoon filters are popular in creative and entertainment applications; however, most filters produce generic effects. The motivation for SpongeBob Inspired was to create something unique a system that captures the personality and style of SpongeBob SquarePants using computer vision techniques. This project combines art and programming, providing an opportunity to learn core image processing methods while developing an interactive system that users can enjoy directly through a web interface.

5. Methodology

The methodology involves a structured image processing pipeline integrated into a web platform:

1. **Image Upload:**
The user can either upload an existing image from their device or capture a new photo directly using the camera through the web interface.
2. **Preprocessing:**
The image is resized and converted into RGB format for consistent color representation.
3. **Bilateral Filtering:**
Smooths image regions while preserving edges, creating the foundation for cartoon-style shading.
4. **Edge Detection (Adaptive Threshold):**
Highlights edges to produce clear outlines resembling cartoon drawings.
5. **Color Quantization:**
Reduces the number of colors using K-means clustering, producing flat, simplified color areas.
6. **SpongeBob Color Palette:**
Applies a customized yellow-based palette inspired by SpongeBob's design.
7. **Intensity Control:**
Uses a slider to let the user adjust the strength of the cartoon effect before generating the final image.
8. **Final Output:**
Combines edges and simplified colors to produce the final cartoonified image displayed through the web interface.



6. System Design

The *SpongeBob Inspired* system includes two main components:

- **Front-End (Web Interface):**
Developed using HTML, CSS, and JavaScript, the front-end allows users to either upload an image or take a photo using their camera, then view the processed cartoon-style result directly online.
- **Back-End (Processing Engine):**
Built in Python using OpenCV and NumPy libraries. The server receives the uploaded image, processes it according to the defined pipeline, and returns the cartoonified output to the browser.

This design provides an easy-to-use platform for non-technical users while maintaining flexibility for parameter adjustments, such as edge strength or color levels.

7. Implementation Details

Below is a simplified Python implementation illustrating the core image processing steps:

1. Streamlit Initialization and Page Setup

```
st.set_page_config(page_title="🧽 SpongeBob-inspired Cartoonize", page_icon="🧽", layout="centered")
```

- Sets the web app's title, favicon, and layout configuration (centered content alignment).

2. File Loading and Base64 Conversion

```
def get_base64_of_bin_file(bin_file):  
    with open(bin_file, 'rb') as f:  
        data = f.read()  
    return base64.b64encode(data).decode()
```

- Converts binary files (e.g., images) into Base64 format for embedding in Streamlit's HTML/CSS.
- Used for loading background images and small page icons.

3. Session State Management

```
if 'theme' not in st.session_state:  
    st.session_state.theme = 'light'  
if 'input_mode' not in st.session_state:  
    st.session_state.input_mode = 'upload'
```

Stores user preferences such as:

- Current theme (light or dark).
- Image input method (upload or camera).

4. Background Music Integration

```
with open("spongmusic.m4a", "rb") as audio_file:
    audio_bytes = audio_file.read()
st.audio(audio_bytes, format='audio/m4a')
```

- Plays background audio when the user interacts with the web.

5. Theme Configuration

```
LIGHT_THEME = { ... }
DARK_THEME = { ... }

def switch_theme():
    st.session_state.theme = 'dark' if st.session_state.theme == 'light' else 'light'
```

- Defines color palettes, backgrounds, and text styles for both light and dark modes.
- Allows switching between themes dynamically.

6. Custom CSS Styling

```
st.markdown(f"<style> ... </style>", unsafe_allow_html=True)
```

Applies advanced styling to the Streamlit interface, including:

- Gradient background and image overlay.
- Upload box appearance and border effects.
- Typography and button styles.
- Animated elements (e.g., pulse effects).

7. Image Input Interface

```
st.button("📷 Upload Image")
st.button("📷 Camera")
```

- Allows the user to choose between uploading an image or capturing one using the device camera.

8. Cartoonization Function (SpongeBob Style)

```
def spongebob_style(frame, intensity):
    # Blur, edge detection, color enhancement, and combination
    return cartoon
```

Performs image processing steps including:

- Image smoothing and bilateral filtering.
- Edge detection for cartoon outlines.
- Color enhancement based on intensity.
- Merging colors and edges to generate the final cartoon image.

9. Image Upload or Camera Capture Process

```
if st.session_state.input_mode == 'upload':  
    uploaded_file = st.file_uploader(...)   
elif st.session_state.input_mode == 'camera':  
    picture = st.camera_input(...)
```

Once the image is provided:

- Displays the original image.
- Allows the user to adjust the cartoon intensity using a slider.
- Converts the image when the button is clicked and displays the cartoon result.
- Provides an option to download the final output using st.download_button.

8. Results and Discussion

The system successfully converts both uploaded and camera-captured photos into SpongeBob-style cartoon images with bright colors, thick outlines, and a cheerful aesthetic .The web interface ensures accessibility and ease of use, allowing users to instantly see their cartoonified results from either uploaded or directly captured images without any complex setup.

The following figures demonstrate the process and output results of the system:



Figure 1, Uploading an image from the device

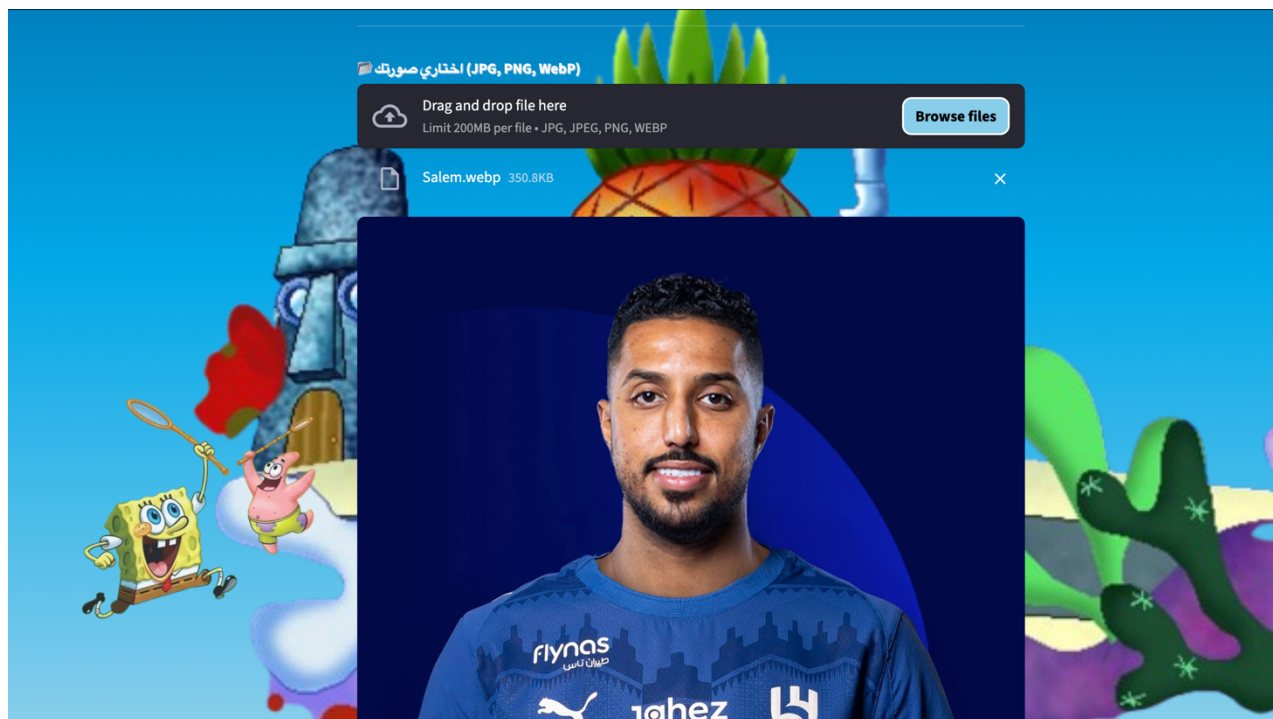


Figure 2, Clicking the “Convert to Cartoon” button to process the uploaded image

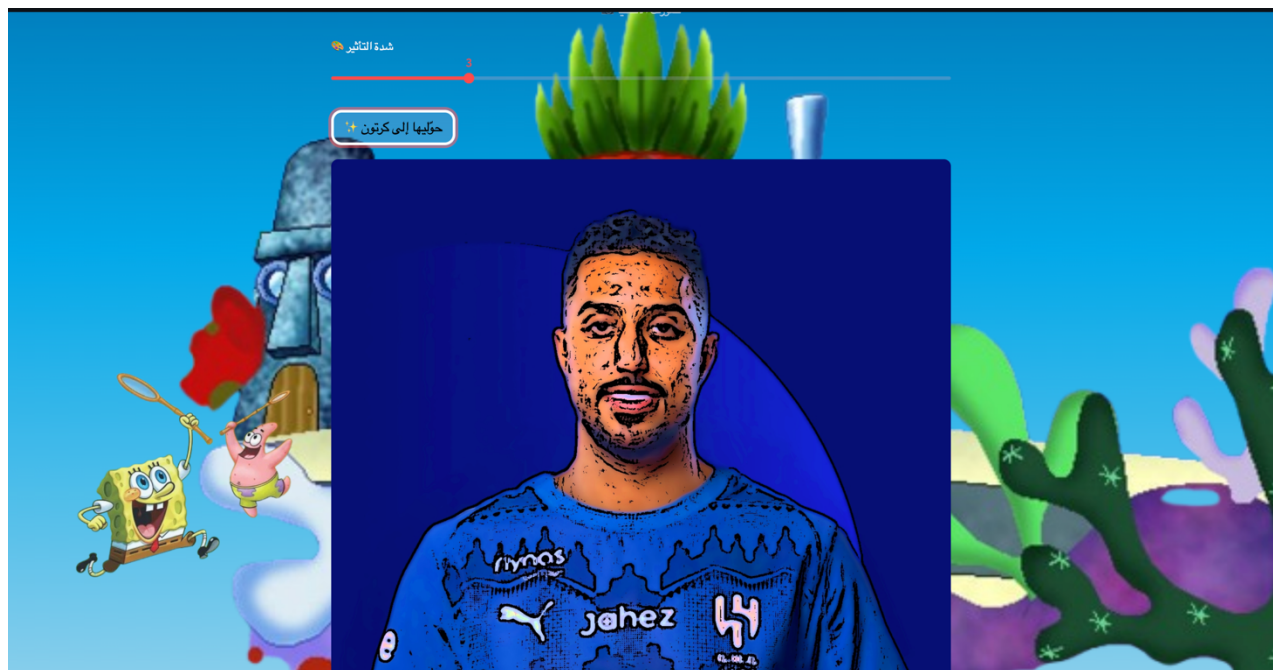


Figure 3, The cartoonified result of the uploaded image and controlling the intensity

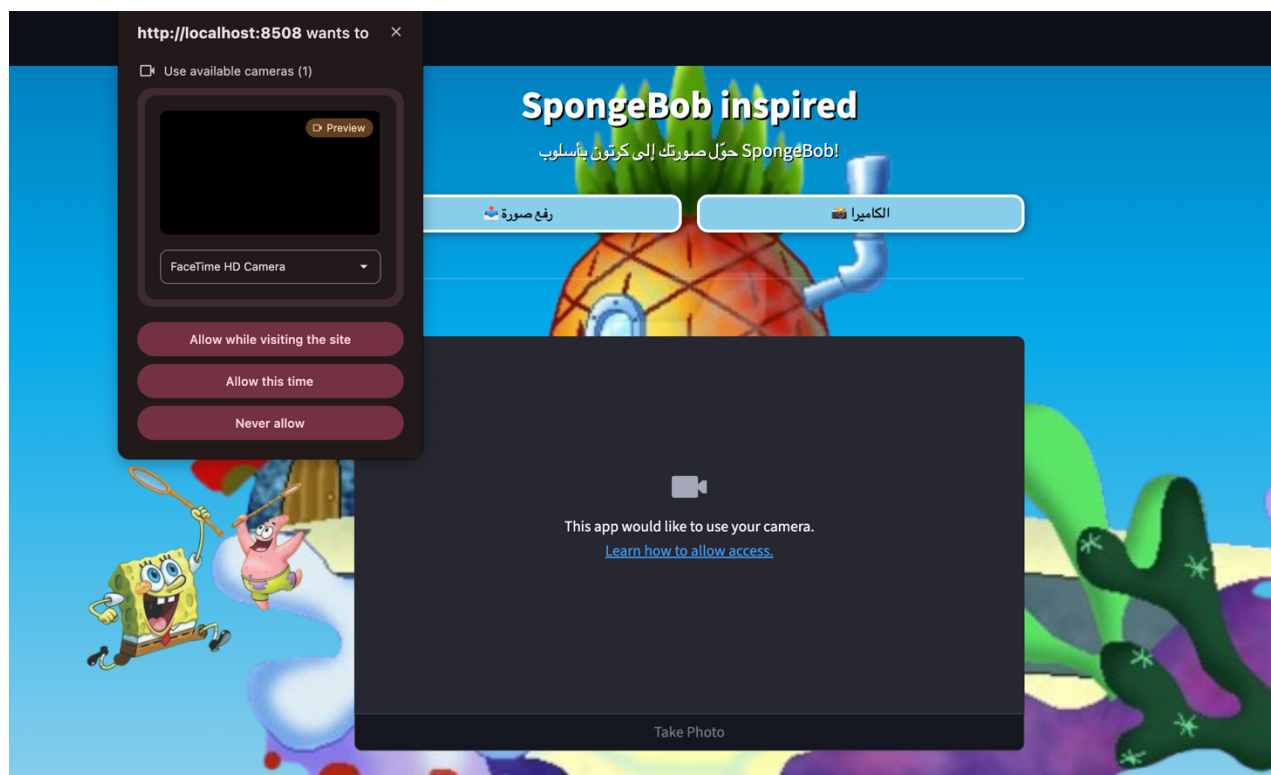


Figure 4, Capturing an image directly from the camera and displaying the cartoonified result.

9. Conclusions and Future Work

9.1 Conclusion:

The SpongeBob Inspired project successfully integrates classical image processing techniques into a functional web application that transforms photos into cartoon-like images. By combining bilateral filtering, edge detection, and color quantization with a SpongeBob-inspired palette, the system produces playful, vibrant results efficiently. The project highlights how technical knowledge can be applied to build fun, interactive tools with real-world appeal.

9.2 Future Work:

Future improvements could include:

- Adding support for mobile devices to enhance accessibility.
- Implementing customizable style themes for other cartoon characters.
- Providing adjustable filters and palette options for user creativity.
- Exploring AI-based style transfer as an optional advanced mode.
- Enabling real-time image transformation similar to Snapchat filters, allowing users to see the cartoon effect instantly through the camera.

References

- [1] C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 839–846, Jan. 1998. [Online]. Available: <https://doi.org/10.1109/ICCV.1998.710815>. [Accessed: Nov. 4, 2025].
- [2] J. Canny, "A computational approach to edge detection," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679–698, Nov. 1986. [Online]. Available: <https://doi.org/10.1109/TPAMI.1986.4767851>. [Accessed: Nov. 4, 2025].
- [3] Y. Chen, J. Shen, and W. Fang, "CartoonGAN: Generative adversarial networks for photo cartoonization," Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9465–9474, Jun. 2018. [Online]. Available: <https://doi.org/10.1109/CVPR.2018.00986>. [Accessed: Nov. 4, 2025].